

Hệ thống Đấu giá Trực tuyến (Online Auction System)

Báo cáo dự án — Ứng dụng client-server đa luồng bằng Java 17

Công nghệ: Java 17 · JavaFX 21 · TCP Socket + JSON (Gson) · MySQL 8 + HikariCP · Maven | **Kiểm thử:** JUnit 5 · Mockito · H2 (36 lớp test, 360 ca kiểm thử) | **Quy mô:** ~80 lớp Java

1. Giới thiệu — Mục tiêu và phạm vi

Dự án xây dựng một **nền tảng đấu giá trực tuyến nhiều người dùng đồng thời**, mô phỏng một sàn đấu giá thực: người bán đăng sản phẩm và mở phiên, người mua đặt giá theo thời gian thực, hệ thống tự quản lý vòng đời phiên và cập nhật tức thì cho tất cả người đang theo dõi. Trọng tâm kỹ thuật là **tính đúng đắn trong môi trường tương tranh (concurrency)** — nhiều luồng cùng đặt giá trên một phiên mà không xảy ra mất tiền, đặt giá "lậu", hay deadlock.

Mục tiêu

- Mô hình client-server thật:** tách hẳn tiến trình client (JavaFX) và server (socket), giao tiếp qua giao thức JSON do nhóm tự định nghĩa thay vì gọi hàm trực tiếp — phản ánh đúng kiến trúc phân tán.
- Đấu giá thời gian thực:** khi có giá mới hoặc phiên đổi trạng thái, mọi client đang theo dõi được **push** cập nhật ngay (server chủ động đẩy, không cần client hỏi lại).
- An toàn dữ liệu khi tương tranh:** số dư người dùng và giá hiện tại của phiên luôn nhất quán dù chịu tải cao; có hợp đồng khóa (lock contract) rõ ràng để tránh deadlock.
- Kiến trúc phân tầng, dễ mở rộng:** áp dụng các mẫu thiết kế (Factory, Observer, DAO, Command, Singleton) và quy tắc phụ thuộc giữa các tầng được tài liệu hóa.

Phạm vi thực hiện

Đã thực hiện:

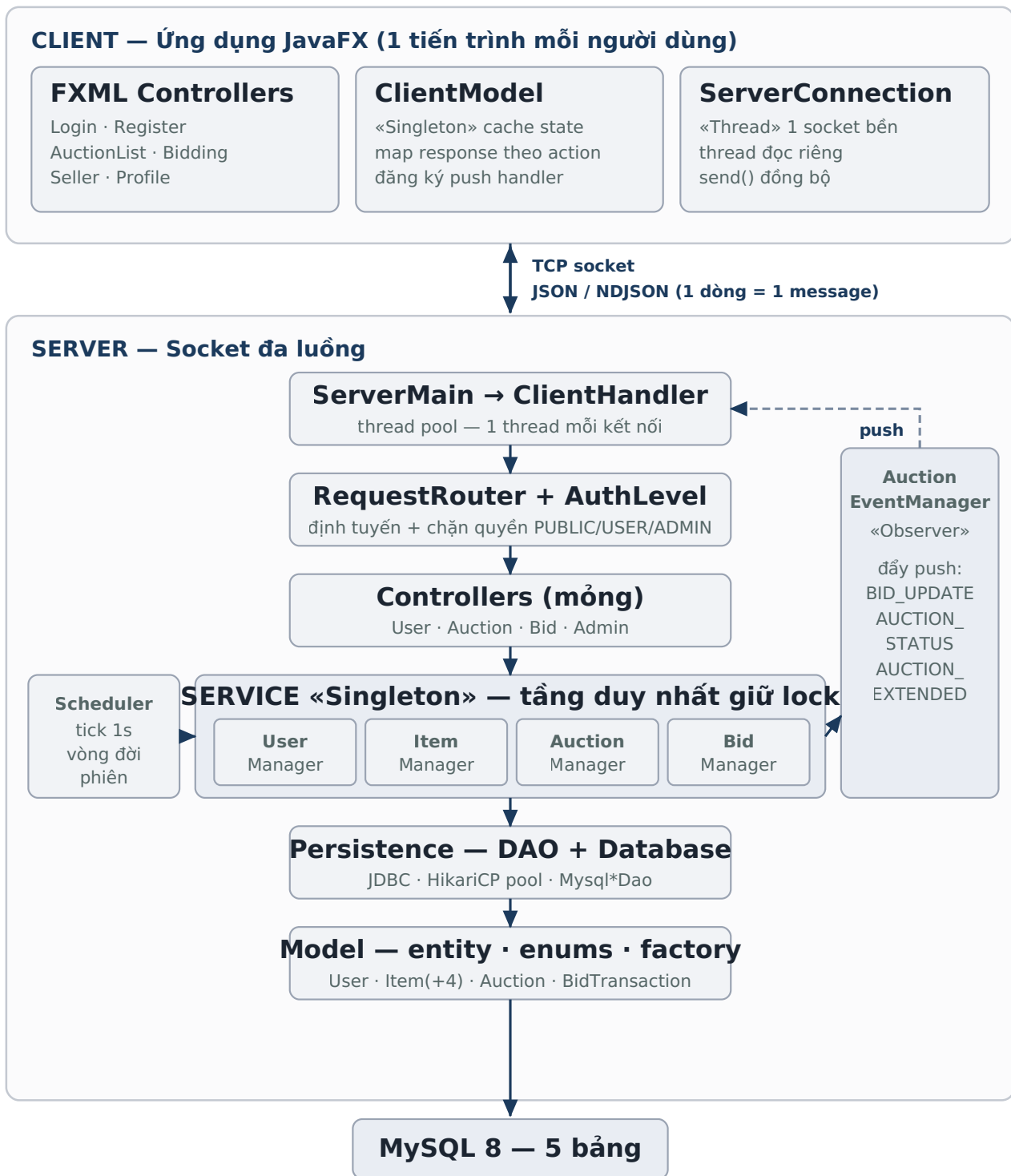
- Đăng ký / đăng nhập, phân quyền 3 cấp (Bidder / Seller / Admin).
- Quản lý sản phẩm 4 nhóm (Art, Electronics, Vehicle, Other) và tạo phiên đấu giá.
- Đặt giá thủ công + **đấu giá tự động (auto-bid)** theo ngưỡng tối đa.
- Anti-sniping:** tự gia hạn phiên khi có giá ở phút cuối.
- Push realtime, upload ảnh, bảng điều khiển Admin, thanh toán / bỏ cọc.

Giới hạn (ngoài phạm vi):

- Chưa tích hợp cổng thanh toán thật — dùng số dư ví nội bộ mô phỏng.
- Phiên auth theo từng kết nối (session per-connection), chưa dùng token JWT (đã chứa sẵn trường `token` để mở rộng).
- Một số điểm yếu đã nhận diện và ghi nhận (đẩy push đồng bộ, đối soát tiền treo) — xem mục 6.

2. Kiến trúc tổng thể

Hệ thống theo kiến trúc **phân tầng (layered) trên nền client-server**. Hai tiến trình tách biệt giao tiếp **chỉ** qua giao thức `protocol` (JSON Request/Response, mỗi dòng một message – NDJSON) trên TCP, không chia sẻ object Java. Phía server, một yêu cầu đi tuần tự qua các tầng: *mạng* → *định tuyến/ủy quyền* → *controller* → *service* (ng nghiệp vụ + khóa) → *persistence* (DAO) → *model*.



Hình 1. Kiến trúc tổng thể: client JavaFX và server socket đa luồng. Yêu cầu đi xuống theo các tầng; sự kiện realtime được đẩy ngược lên client qua Observer.

Mô tả theo kiến trúc

Client Ứng dụng JavaFX gồm các **FXML Controller** chỉ lo hiển thị/binding; mọi trạng thái dùng chung nằm ở **ClientModel** (Singleton). **ServerConnection** giữ một socket bền và một thread đọc riêng, nên UI không bao giờ bị "đứng" do chờ mạng — các lời gọi blocking đi qua hàng đợi + callback của ClientModel.

Server **ServerMain** mở socket lắng nghe và giao mỗi kết nối cho một **ClientHandler** chạy trên thread pool (mặc định 100 thread). ClientHandler đọc JSON, chuyển cho **RequestRouter** — nơi vừa định tuyến `action → handler` vừa làm **middleware uỷ quyền**: mỗi action khai báo mức

quyền tối thiểu (PUBLIC / USER / ADMIN) và bị chặn trước khi vào controller. Controller rất mỏng, chỉ chuyển tiếp sang **Service**.

Service Đây là tầng nghiệp vụ và là **tầng duy nhất được phép mua lock**. **BidManager** điều phối đặt giá, **AuctionManager** quản lý vòng đời phiên (qua một scheduler tick mỗi giây) và anti-sniping, **UserManager** / **ItemManager** lo tài khoản và sản phẩm. Các manager là Singleton, dùng `ConcurrentHashMap` làm cache để tránh khóa toàn cục.

Persistence Tầng DAO đóng gói toàn bộ JDBC sau các interface `*Dao` với bản cài đặt `Mysql*Dao`; kết nối qua **HikariCP** pool. DAO chỉ làm CRUD, không chứa nghiệp vụ.

Model Các entity domain thuần giữ bất biến nội tại của chính mình: `User.tryReserve` trừ số dư nguyên tử, `Auction.placeBid` tự mua lock của phiên để kiểm tra-và-cập-nhật giá. **Observer chain** luôn được kích hoạt **sau khi đã nhả lock**.

Luồng đặt giá (PLACE_BID) — minh họa thứ tự khóa chống deadlock

Hợp đồng khóa chính tắc: L2(User người đặt) → L1(Auction) → L2(User leader cũ) → L3(ClientHandler.send). Tại không thời điểm nào một thread giữ đồng thời hai khóa khác loại lồng nhau — đây là điều kiện đủ để hệ thống không thể deadlock.

1. **Giữ chỗ tiền:** `User.tryReserve(amount)` — kiểm tra-và-trừ số dư nguyên tử (synchronized).
Reserve trước khi vào lock của phiên.
2. **Cập nhật phiên:** mua `Auction.lock` (ReentrantLock), kiểm tra hợp lệ, chụp lại leader cũ, cập nhật giá & người dẫn đầu, rồi **nhả lock**.
3. **Thông báo NGOÀI lock:** kích hoạt observer chain → anti-sniping kiểm tra & gia hạn → đẩy push `BID_UPDATE` tới subscriber.
4. **Hoàn tiền leader cũ:** `User(leaderCũ).release(prevAmount)` — trả lại số dư đã giữ chỗ.
5. **Auto-bid:** `tryTriggerAutoBids()` đọc danh sách lock-free; `ThreadLocal` chặn bùng nổ đệ quy khi auto-bid kích hoạt auto-bid khác.

An toàn tiền: nếu lưu DB thất bại sau khi đã giữ chỗ, hệ thống release lại RAM rồi mới ném lỗi — không để lại "ghost reservation".

3. Các chức năng đạt được (theo barem điểm)

Mỗi chức năng nêu rõ hướng giải quyết, lý do lựa chọn và mã action tương ứng.

Chức năng	Hướng giải quyết	Lý do lựa chọn	Action
Tài khoản & phân quyền	Đăng ký/dăng nhập; mật khẩu hash SHA-256 + salt ngẫu nhiên 16 byte (SecureRandom), so khớp bằng <code>MessageDigest.isEqual</code> . Phân quyền 3 vai trò gate ở RequestRouter qua <code>AuthLevel</code> .	So khớp hằng-thời-gian chống timing attack; salt riêng từng user chống rainbow-table. Gate tập trung 1 chỗ giúp giảm tải cho controller.	LOGIN, REGISTER, GET_PROFILE
Quản lý sản phẩm đa loại	4 nhóm (Art, Electronics, Vehicle, Other) kế thừa lớp <code>Item</code> ; tạo qua Factory Method (<code>ItemFactory.create</code>) dựa theo <code>ItemCategory</code> với thuộc tính riêng từng loại.	Factory tách thao tác new khỏi service: thêm loại sản phẩm mới chỉ sửa Factory, không đụng phần gọi → tuân thủ nguyên lý Open/Closed.	CREATE_ITEM, LIST_ITEMS
Tạo & quản lý phiên đấu giá	Seller mở phiên từ item. Scheduler tick mỗi giây tự chuyển trạng thái <code>PENDING→RUNNING→FINISHED</code> ; trạng thái chuyển đổi tuần tự nghiêm ngặt.	Vòng đời do server tự lái đảm bảo phiên luôn đóng đúng hạn; máy trạng thái tường minh chặn chuyển trạng thái sai.	CREATE_AUCTION, CLOSE_AUCTION
Đặt giá tương tranh	Quy trình giữ-chỗ-tiền → mua lock phiên → cập nhật → nhả lock → thông báo, theo hợp đồng khóa cố định. Validate giá $\geq$ giá hiện tại + bước tối thiểu; chặn seller tự bid.	Thứ tự khóa nhất quán ngăn deadlock; reserve trước lock giữ số dư luôn nhất quán dưới tải cao (đã kiểm thử 100 thread đồng thời).	PLACE_BID, BID_HISTORY
Đấu giá tự động (Auto-bid)	Người dùng đặt giá tối đa + bước nhảy ; hệ thống tự thay họ đặt tới mức tối thiểu cần thiết. <code>ThreadLocal</code> guard chặn đệ quy vô hạn giữa nhiều auto-bid.	Mô phỏng đúng cơ chế proxy-bid của các sàn thật, tăng tính cạnh tranh mà người dùng không phải canh thủ công.	SET_AUTO_BID

Chức năng	Hướng giải quyết	Lý do lựa chọn	Action
Anti-sniping (phút chót)	Nếu giá mới rơi vào thời gian cuối, phiên tự gia hạn thêm thời gian cấu hình. Cài qua một <i>internal observer</i> của <code>AuctionManager</code> , gọi <code>Auction.extend</code> và push <code>AUCTION_EXTENDED</code> .	Đảm bảo công bằng: người trả giá ở giây cuối không tự động thắng vì hết giờ. Tham số cấu hình linh hoạt qua file <code>.env</code> .	(push) <code>AUCTION_EXTENDED</code>
Realtime push	Mẫu Observer : client <code>WATCH_AUCTION</code> để đăng ký; <code>AuctionEventManager</code> fan-out sự kiện tới các <code>ClientHandler</code> đang theo dõi. <code>Disconnect</code> → tự dọn dẹp subscription.	Server chủ động đẩy thay vì client polling → độ trễ thấp, giảm tải cho server. Không cần đăng nhập vẫn xem được biến động giá.	<code>WATCH_AUCTION</code> , (push) <code>BID_UPDATE</code>
Thanh toán & bỏ cọc	Winner <code>CONFIRM_PAYMENT</code> để trả tiền hoặc <code>FORFEIT_AUCTION</code> để bỏ cọc. Quá hạn, scheduler tự xử lý hộ. Cổng xử lý nguyên tử trạng thái chống race condition.	Mô phỏng nghĩa vụ thanh toán thật và xử lý người thắng huỷ kèo; cổng atomic tránh race khi tác vụ tự động và người dùng trùng nhau.	<code>CONFIRM_PAYMENT</code> , <code>FORFEIT_AUCTION</code>
Bảng điều khiển Admin	Liệt kê user (ẩn thông tin nhạy cảm), khóa/mở khóa tài khoản, đóng phiên cưỡng chế. Action gắn chặt mức quyền <code>AuthLevel.ADMIN</code> .	Tách quyền quản trị rõ ràng; lọc dữ liệu nghiêm ngặt ở tầng trả về để không rò rỉ thông tin bảo mật.	<code>LIST_USERS</code> , <code>BAN_USER</code>
Upload & phục vụ ảnh	Ảnh truyền chuỗi Base64; server whitelist đuôi (<code>jpg/png/gif/webp</code>), giới hạn 3 MB, lưu cục bộ và chặn path-traversal khi đọc lại. Client có cache ảnh.	Sử dụng whitelist và giới hạn dung lượng để phòng thủ tối thiểu; cơ chế cache giúp giảm tải băng thông và độ trễ.	<code>UPLOAD_IMAGE</code> , <code>GET_IMAGE</code>

4. Mẫu thiết kế & điểm nhấn kỹ thuật

- **Singleton:** 4 service manager giữ state toàn cục (Bill-Pugh), thread-safe.
- **Factory Method:** `ItemFactory` đóng gói logic khởi tạo thực thể đa hình.
- **Observer:** Chuỗi sự kiện realtime xử lý độc lập bên ngoài phân vùng lock.
- **Command:** Chuẩn hóa logic xử lý gói tin qua từng `CommandHandler`.
- **Lock-free structures:** Tối ưu hiệu năng thông qua cấu trúc `ThreadLocal` và `Atomic`.
- **Hợp đồng khóa cố định:** Tài liệu hóa thứ tự và quy tắc lỏng khóa để triệt tiêu deadlock.
- **Độ chính xác tài chính:** Toàn bộ trường tiền tệ sử dụng kiểu dữ liệu `BigDecimal`.
- **Cấu hình tập trung:** Quản trị tập trung các hằng số hệ thống qua tệp `.env`.

5. Kiểm thử & chất lượng

Dự án tích hợp **36 lớp test với 360 ca kiểm thử** (JUnit 5 + Mockito + H2 in-memory), phủ rộng toàn bộ thực thể, nghiệp vụ service, tầng DAO, router và controller. Các kịch bản kiểm thử tương tranh trực tiếp xác nhận tính đúng đắn của hợp đồng khóa: khi 100 luồng đồng thời thực hiện

thao tác đặt giá trên cùng một phiên, hệ thống đảm bảo ghi nhận chính xác duy nhất một lịch sử bid hợp lệ và số dư tài khoản được khấu trừ đồng bộ tuyệt đối.

6. Hạn chế đã nhận diện & hướng phát triển

- **Khối đẩy push đồng bộ:** Tiến trình ghi socket hiện tại thực hiện trực tiếp trong chuỗi thông báo, một client kết nối kém có thể ảnh hưởng gián tiếp đến toàn hệ thống. *Biện pháp:* Chuyển sang mô hình writer bất đồng bộ sử dụng hàng đợi chuyên dụng.
- **Xử lý định danh gói tin:** Phản hồi phía client đang map theo tên `action`, dễ gây xung đột khi chạy song song. *Biện pháp:* Chuyển cấu trúc định danh dựa trên trường `requestId` (UUID).

Kết luận

Hệ thống đáp ứng trọn vẹn và hoàn chỉnh các tính năng cốt lõi của một nền tảng đấu giá trực tuyến chuyên nghiệp trên nền tảng kiến trúc phân tầng client-server rõ ràng. Điểm sáng và giá trị kỹ thuật cốt lõi của dự án nằm ở phương pháp giải quyết **bài toán tương tranh có kiểm soát**: các điều khoản khóa được định nghĩa và thiết kế một cách tường minh, được chứng thực qua hệ thống kiểm thử đa luồng nghiêm ngặt, kết hợp nhịp nhàng cùng các cấu trúc dữ liệu tối ưu nhằm mang lại tính toàn vẹn dữ liệu cao nhất và hiệu năng vận hành tối đa.